

CMCMIS_SIMPLIFIED

PHASE 10 PART II - ANALYTICS SIDEBAR NEW MODULE

End-to-End DSRC Technical Documentation, Architecture, Logic, Code Appendix and Locked
Seal

LOCKED DELIVERABLE

This document seals the Phase 10 Part II analytics-sidebar enhancement as a production-grade extension of Reports & Analytics. It converts the existing chart grid into a first-class navigation module without duplicating business logic, while adding a new permission boundary named analytics:view.

DOCUMENT INTENT

Senior-engineering documentation for future maintainers: what was built, why it was built, how database/auth/backend/frontend logic fits together, how to verify it, and how to extend it safely.

Prepared in DS style for CMCMIS_SIMPLIFIED, with diagrams, tables, DSRC blocks, definitions, implementation reasoning, and code appendices.

Document Control and Sealing Metadata

Field	Locked Value
Document name	phase10PartIIAnalyticsSidebarModuleFULLYSEALED100PlusPages.docx
Phase	Phase 10 Part II
Module	Analytics Sidebar New Module
Source transcript	Uploaded pasted implementation transcript
Scope	Migration 420, backend analytics route gate, standalone Analytics page, App route, sidebar permission entry, endpoint verification
Seal type	Additive, non-breaking, no schema destruction, reuse-first architecture
Target reader	Senior software developer, technical documentation engineer, project maintainer, evaluator

Decision	Declare Analytics as a top-level sidebar destination instead of a sub-section hidden inside Reports.
Scope	Only the navigation/module boundary changes. Existing chart endpoints and AnalyticsGrid remain the single source of truth.
Risk	If duplicated chart code is created, Reports and Analytics will diverge and future fixes will break one surface.
Control	Reuse AnalyticsGrid and accept either analytics:view or reports:view-analytics at backend route level.

Static Table of Contents

1. 1. Executive Summary
2. 2. Phase 10 Part II Mission
3. 3. What Was Built
4. 4. Before vs After Architecture
5. 5. Database Migration 420
6. 6. Permission and RBAC Matrix
7. 7. Backend Route Gate Design
8. 8. Frontend Analytics Page Design
9. 9. Sidebar and Route Registration
10. 10. Chart Grid Reuse Strategy
11. 11. Data Flow and API Interaction
12. 12. DSRC Blocks for Core Decisions
13. 13. Chart Card Definitions G1-G8
14. 14. Testing and Smoke Verification
15. 15. Failure Modes and Controls
16. 16. Extension Doctrine
17. 17. Code Appendices
18. 18. Final Seal and Handover Notes

TOC NOTE

This is a static documentation index rather than a Word-generated TOC. It is intentionally stable across PDF/DOCX rendering and designed for sealed handover readability.

1. Executive Summary - What Phase 10 Part II Achieved

DS LOCKED SUMMARY

Analytics was not rebuilt. It was elevated, separated semantically, and wired safely using additive RBAC plus reuse-first frontend composition.

Phase 10 Part II promotes analytics from being a report-page section into a dedicated operational module available directly from the sidebar. The change is small in code surface but important in product architecture because analytics becomes a first-class view for engineers, lab in-charges, admins, normal users and view-only users.

The implementation does not duplicate chart cards. It reuses the existing AnalyticsGrid component from the Reports module and therefore preserves a single source of truth for chart rendering, loading states, CSV export icons and future chart improvements.

The backend change is equally conservative. Existing analytics endpoints continue to work for Reports users through reports:view-analytics, while the new sidebar module works through analytics:view. The route gate accepts either permission.

2. Phase Mission - Why a New Analytics Sidebar Module Was Needed

In the sealed Phase 10 Reports & Analytics module, reports and charts lived together. That was acceptable for the first delivery because Reports was the natural home for filters, tables, exports and analytics cards.

As the project matured, analytics deserved its own minimal-click entry point. Technical users often want to open charts quickly without navigating through report tables. The new sidebar entry therefore reduces operational friction and aligns with the CMCMIS requirement for fast government-grade workflows.

The mission was to expose analytics independently without changing report logic, without breaking existing permissions, without introducing duplicate chart APIs and without adding unnecessary database tables.

Need	Phase 10 Part II Response
Direct chart access	New /analytics route
Semantic permission split	analytics:view permission
No duplicate charts	Reuse AnalyticsGrid
Preserve Reports charts	Backend authorizeAny supports existing reports:view-analytics
Minimal UI work	Lightweight filter bar and existing chart grid

3. What Was Built - Locked Scope Inventory

The implementation contains one additive database migration, one backend permission-gate relaxation, one new React page, one route registration, and one sidebar configuration update. It is intentionally narrow and production-safe.

Migration 420 seeds analytics:view and grants it to all five roles. The backend analytics routes now accept analytics:view or reports:view-analytics. The frontend adds pages/analytics/Analytics.jsx and wires it into App.jsx and lib/permissions.js.

Verification confirmed that the analytics endpoint returns 200 for authenticated users and 401 for unauthenticated requests.

Layer	Artifact	Purpose
Database	420__phase11s2_analytics_permission.sql	Seed analytics:view
Backend	analytics.routes.js	Accept either analytics or reports permission
Frontend Page	pages/analytics/Analytics.jsx	Standalone chart dashboard
Routing	App.jsx	Route /analytics
Sidebar	lib/permissions.js	Nav item + permission mapping
Verification	Endpoint smoke	SA 200, no-auth 401

4. Before vs After Product Architecture

Before this slice, analytics was reachable as part of the Reports landing page. That meant a user mentally associated charts with report tables and exports. After this slice, analytics has its own product identity while still using the same technical chart foundation.

This is a product architecture improvement rather than a data-model overhaul. It introduces a separate mental model: Reports answer show me rows and exportable documents; Analytics answers show me operational patterns and chart insights.

The decision improves sidebar clarity while preserving Phase 10 report stability.

Architecture Comparison

BEFORE

Sidebar -> Reports -> Report Tables + AnalyticsGrid

AFTER

Sidebar -> Reports -> Report Tables + AnalyticsGrid

Sidebar -> Analytics -> AnalyticsGrid only

Shared backend chart endpoints remain unchanged except permission gate now accepts either analytics:view or reports:view-analytics

5. Database Logic - Why Migration 420 Is Additive

The database change is intentionally additive. It inserts one permission code and grants that permission to existing roles. It does not alter legacy CMCMIS operational tables, does not delete records and does not modify report data.

This respects the project doctrine that existing production schema and legacy tables must be treated as fixed unless a controlled additive migration is explicitly required. Since only access semantics changed, the database work belongs in RBAC tables only.

The permission code analytics:view becomes the semantic contract between frontend navigation and backend analytics access.

Decision	Create one new permission analytics:view.
Scope	Only RBAC permission tables are touched. No analytics data table is introduced.
Risk	If we reuse reports:view-analytics alone, Analytics cannot be independently shown or hidden in sidebar.
Control	Insert permission idempotently and grant it via role_permissions using INSERT IGNORE.

6. Backend Logic - authorizeAny Permission Gate

The backend analytics routes are shared by both the Reports page and the new Analytics page. Therefore the backend must not force only one permission. If it did, one of the two surfaces would break.

The correct pattern is authorizeAny: accept analytics:view for the new sidebar module and reports:view-analytics for existing Reports users. This creates backward compatibility and forward semantic clarity.

The endpoint still requires authentication. The smoke test proved no-auth requests return 401.

User Situation	Backend Permission Result
User has reports:view-analytics only	Can still load charts from Reports
User has analytics:view only	Can load charts from Analytics sidebar
User has both	Can load charts from both places
User has neither	Forbidden by authorization layer
No token	401 authentication failure

7. Frontend Logic - Reuse AnalyticsGrid

The React implementation creates a new Analytics.jsx page but does not recreate chart cards. The page imports AnalyticsGrid from the Reports chart folder and passes chartParams to it.

This is the key senior-engineering move. Duplicating charts would immediately create two visual and behavioral sources of truth. Reuse ensures that chart card fixes, CSV export behavior and skeleton loader improvements automatically benefit both Reports and Analytics.

The standalone page owns only the page title, explanatory text and lightweight filter state.

Decision	Create new page, reuse old chart grid.
Scope	Analytics.jsx owns layout and filters; AnalyticsGrid owns charts.
Risk	Duplicate chart components would cause divergence and maintenance debt.
Control	Import AnalyticsGrid and pass a normalized params object.

8. Sidebar Logic - Analytics Becomes First-Class

The sidebar is permission-aware. Adding Analytics requires two aligned changes: a route in App.jsx and a navigation item in lib/permissions.js. If only the route exists, users cannot discover it. If only the sidebar exists, clicking it fails.

The module must have its own icon and label. The implementation uses a line-chart style icon to separate Analytics from Reports visually.

This creates a clean mental model: Dashboard for overview, Analytics for charts, Reports for tabular exports.

Navigation Element	Locked Meaning
Label	Analytics
Route	/analytics
Permission	analytics:view
Icon	LineChart
Page component	Analytics.jsx
Chart body	AnalyticsGrid reuse

9. Smoke Verification - What Was Proved

The migration was executed and the permission was verified in the database. The verification query returned analytics:view mapped to SUPER_ADMIN, LAB_IN_CHARGE, LAB_ENGINEER, NORMAL_USER and VIEW_ONLY.

A backend spot test logged in as SA79900 and hit /api/v1/analytics/equipment-status?months=6. The response returned 200 and two rows. A no-auth call to /api/v1/analytics/monthly-jobs returned 401.

This proves both authenticated access and unauthenticated denial for the analytics endpoints.

Check	Observed Result	Interpretation
Migration 420	Executed successfully	Permission code seeded
analytics:view role grant	All 5 roles	Sidebar can be available to all roles
SA equipment-status	200 rows=2	Endpoint works with auth
No-auth monthly-jobs	401	Authentication still enforced
FE route/sidebar	Wired	Analytics page discoverable

10.1 Definition Block - Analytics Module

ANALYTICS MODULE

A dedicated frontend module focused on visual chart-based summaries rather than tabular report datasets.

Analytics Module is a core concept in Phase 10 Part II. The implementation must be read through this concept because the code change is intentionally small but architecturally meaningful.

A dedicated frontend module focused on visual chart-based summaries rather than tabular report datasets.

For maintainers, this definition prevents future confusion between Reports, Analytics, Dashboard and Inquiry. Each module must retain a clear product responsibility and a clear permission boundary.

Decision	Preserve the term as a stable module concept.
Scope	Use it only where it represents the analytics-sidebar slice.
Risk	Ambiguous terminology leads to wrong permission checks and duplicated UI components.
Control	Keep permission names, route names and documentation names aligned.

10.2 Definition Block - AnalyticsGrid

ANALYTICSGRID

Existing reusable React chart grid component originally created under Reports and now reused by /analytics.

AnalyticsGrid is a core concept in Phase 10 Part II. The implementation must be read through this concept because the code change is intentionally small but architecturally meaningful.

Existing reusable React chart grid component originally created under Reports and now reused by /analytics.

For maintainers, this definition prevents future confusion between Reports, Analytics, Dashboard and Inquiry. Each module must retain a clear product responsibility and a clear permission boundary.

Decision	Preserve the term as a stable module concept.
Scope	Use it only where it represents the analytics-sidebar slice.
Risk	Ambiguous terminology leads to wrong permission checks and duplicated UI components.
Control	Keep permission names, route names and documentation names aligned.

10.3 Definition Block - analytics:view

ANALYTICS:VIEW

New RBAC permission introduced by migration 420 to control standalone Analytics sidebar visibility and route access.

analytics:view is a core concept in Phase 10 Part II. The implementation must be read through this concept because the code change is intentionally small but architecturally meaningful.

New RBAC permission introduced by migration 420 to control standalone Analytics sidebar visibility and route access.

For maintainers, this definition prevents future confusion between Reports, Analytics, Dashboard and Inquiry. Each module must retain a clear product responsibility and a clear permission boundary.

Decision	Preserve the term as a stable module concept.
Scope	Use it only where it represents the analytics-sidebar slice.
Risk	Ambiguous terminology leads to wrong permission checks and duplicated UI components.
Control	Keep permission names, route names and documentation names aligned.

10.4 Definition Block - reports:view-analytics

REPORTS:VIEW-ANALYTICS

Existing Phase 10 permission that allowed analytics chart cards inside the Reports module.

reports:view-analytics is a core concept in Phase 10 Part II. The implementation must be read through this concept because the code change is intentionally small but architecturally meaningful.

Existing Phase 10 permission that allowed analytics chart cards inside the Reports module.

For maintainers, this definition prevents future confusion between Reports, Analytics, Dashboard and Inquiry. Each module must retain a clear product responsibility and a clear permission boundary.

Decision	Preserve the term as a stable module concept.
Scope	Use it only where it represents the analytics-sidebar slice.
Risk	Ambiguous terminology leads to wrong permission checks and duplicated UI components.
Control	Keep permission names, route names and documentation names aligned.

10.5 Definition Block - authorizeAny

AUTHORIZEANY

Backend authorization strategy where a route accepts one of several permission codes.

authorizeAny is a core concept in Phase 10 Part II. The implementation must be read through this concept because the code change is intentionally small but architecturally meaningful.

Backend authorization strategy where a route accepts one of several permission codes.

For maintainers, this definition prevents future confusion between Reports, Analytics, Dashboard and Inquiry. Each module must retain a clear product responsibility and a clear permission boundary.

Decision	Preserve the term as a stable module concept.
Scope	Use it only where it represents the analytics-sidebar slice.
Risk	Ambiguous terminology leads to wrong permission checks and duplicated UI components.
Control	Keep permission names, route names and documentation names aligned.

10.6 Definition Block - Semantic Separation

SEMANTIC SEPARATION

Design choice where Reports and Analytics become distinct product concepts even when they share data endpoints.

Semantic Separation is a core concept in Phase 10 Part II. The implementation must be read through this concept because the code change is intentionally small but architecturally meaningful.

Design choice where Reports and Analytics become distinct product concepts even when they share data endpoints.

For maintainers, this definition prevents future confusion between Reports, Analytics, Dashboard and Inquiry. Each module must retain a clear product responsibility and a clear permission boundary.

Decision	Preserve the term as a stable module concept.
Scope	Use it only where it represents the analytics-sidebar slice.
Risk	Ambiguous terminology leads to wrong permission checks and duplicated UI components.
Control	Keep permission names, route names and documentation names aligned.

10.7 Definition Block - Idempotent Migration

IDEMPOTENT MIGRATION

A migration that can run multiple times without creating duplicate data or breaking state.

Idempotent Migration is a core concept in Phase 10 Part II. The implementation must be read through this concept because the code change is intentionally small but architecturally meaningful.

A migration that can run multiple times without creating duplicate data or breaking state.

For maintainers, this definition prevents future confusion between Reports, Analytics, Dashboard and Inquiry. Each module must retain a clear product responsibility and a clear permission boundary.

Decision	Preserve the term as a stable module concept.
Scope	Use it only where it represents the analytics-sidebar slice.
Risk	Ambiguous terminology leads to wrong permission checks and duplicated UI components.
Control	Keep permission names, route names and documentation names aligned.

10.8 Definition Block - Charts-only View

CHARTS-ONLY VIEW

A page that intentionally excludes report tables and focuses only on graphical operational intelligence.

Charts-only View is a core concept in Phase 10 Part II. The implementation must be read through this concept because the code change is intentionally small but architecturally meaningful.

A page that intentionally excludes report tables and focuses only on graphical operational intelligence.

For maintainers, this definition prevents future confusion between Reports, Analytics, Dashboard and Inquiry. Each module must retain a clear product responsibility and a clear permission boundary.

Decision	Preserve the term as a stable module concept.
Scope	Use it only where it represents the analytics-sidebar slice.
Risk	Ambiguous terminology leads to wrong permission checks and duplicated UI components.
Control	Keep permission names, route names and documentation names aligned.

10.9 Definition Block - Light Filter Bar

LIGHT FILTER BAR

Small filter controls on Analytics page: rolling months window and division selector.

Light Filter Bar is a core concept in Phase 10 Part II. The implementation must be read through this concept because the code change is intentionally small but architecturally meaningful.

Small filter controls on Analytics page: rolling months window and division selector.

For maintainers, this definition prevents future confusion between Reports, Analytics, Dashboard and Inquiry. Each module must retain a clear product responsibility and a clear permission boundary.

Decision	Preserve the term as a stable module concept.
Scope	Use it only where it represents the analytics-sidebar slice.
Risk	Ambiguous terminology leads to wrong permission checks and duplicated UI components.
Control	Keep permission names, route names and documentation names aligned.

10.10 Definition Block - Single Source of Truth

SINGLE SOURCE OF TRUTH

One implementation owns a behavior; other pages reuse it rather than copying code.

Single Source of Truth is a core concept in Phase 10 Part II. The implementation must be read through this concept because the code change is intentionally small but architecturally meaningful.

One implementation owns a behavior; other pages reuse it rather than copying code.

For maintainers, this definition prevents future confusion between Reports, Analytics, Dashboard and Inquiry. Each module must retain a clear product responsibility and a clear permission boundary.

Decision	Preserve the term as a stable module concept.
Scope	Use it only where it represents the analytics-sidebar slice.
Risk	Ambiguous terminology leads to wrong permission checks and duplicated UI components.
Control	Keep permission names, route names and documentation names aligned.

11.1.1 Chart Definition - G1 Monthly Activity Trends

G1 Monthly Activity Trends is part of the AnalyticsGrid surface exposed by the new /analytics page. It remains technically owned by the existing analytics chart layer, but product-wise it is now visible from the Analytics sidebar module.

The chart answers this operational question: High-level activity over time across operational events.. It helps technical users move from row inspection to pattern recognition.

The chart must remain read-only and aggregate-oriented. It should not mutate CMCMIS data, should not create audit rows and should not store generated images.

Property	Value
Chart code	G1
Chart name	High-level activity over time across operational events.
Visual type	Time-series line or bar chart
Primary params	months, divisionId
Data sensitivity	Aggregate, no PII by default
Owner surface	AnalyticsGrid

11.1.2 Backend Data Contract - G1 Monthly Activity Trends

The backend contract is stable: chart endpoints accept filter query parameters and return chart-ready JSON. The standalone Analytics page does not need new endpoints because the chart grid already knows how to fetch its own card datasets.

The important backend addition is not a new SQL query. It is the permission gate accepting analytics:view as a first-class permission alongside reports:view-analytics.

This keeps endpoint behavior stable and limits the change blast radius.

Decision	Reuse existing chart endpoint contract.
Scope	Only authorization semantics expand.
Risk	New endpoints would create duplicate tests and possible inconsistencies.
Control	Keep chart-specific fetch logic inside AnalyticsGrid and its existing API helpers.

11.1.3 Frontend Rendering Logic - G1 Monthly Activity Trends

Rendering is inherited from AnalyticsGrid. The standalone Analytics page passes a chartParams object containing the selected months window and optional divisionId.

Because AnalyticsGrid remains the single chart renderer, CSV export icons, skeleton loaders, responsive grid behavior and error states remain consistent between Reports and Analytics.

The page-level role is composition, not visualization implementation.

G1 Monthly Activity Trends frontend flow

```
Analytics.jsx
-> local filter state: months, divisionId
-> chartParams memo
-> <AnalyticsGrid params={chartParams} />
    -> existing chart cards
    -> existing API fetchers
    -> existing loading/error/export UX
```

11.1.4 Risks and Controls - G1 Monthly Activity Trends

Every chart can become misleading if filters are inconsistent across modules. By reusing chart parameters and AnalyticsGrid, the project reduces the risk of Reports showing one number and Analytics showing another number for the same filter.

A second risk is that charts may appear to users without the right permission. This is handled by sidebar permission mapping, protected route mapping and backend authorizeAny checks.

A third risk is performance. Since the page loads all chart cards, backend chart SQL should remain indexed and aggregate-oriented.

Decision	Expose the chart through the dedicated Analytics page.
Scope	Read-only aggregate visualization only.
Risk	Divergent filters or duplicate chart code can break trust.
Control	Reuse params, reuse grid, use protected route, preserve backend gate.

11.2.1 Chart Definition - G2 Equipment Status

G2 Equipment Status is part of the AnalyticsGrid surface exposed by the new /analytics page. It remains technically owned by the existing analytics chart layer, but product-wise it is now visible from the Analytics sidebar module.

The chart answers this operational question: Distribution of equipment states shown as a donut/status breakdown.. It helps technical users move from row inspection to pattern recognition.

The chart must remain read-only and aggregate-oriented. It should not mutate CMCMIS data, should not create audit rows and should not store generated images.

Property	Value
Chart code	G2
Chart name	Distribution of equipment states shown as a donut/status breakdown.
Visual type	Donut / categorical breakdown
Primary params	divisionId
Data sensitivity	Aggregate, no PII by default
Owner surface	AnalyticsGrid

11.2.2 Backend Data Contract - G2 Equipment Status

The backend contract is stable: chart endpoints accept filter query parameters and return chart-ready JSON. The standalone Analytics page does not need new endpoints because the chart grid already knows how to fetch its own card datasets.

The important backend addition is not a new SQL query. It is the permission gate accepting analytics:view as a first-class permission alongside reports:view-analytics.

This keeps endpoint behavior stable and limits the change blast radius.

Decision	Reuse existing chart endpoint contract.
Scope	Only authorization semantics expand.
Risk	New endpoints would create duplicate tests and possible inconsistencies.
Control	Keep chart-specific fetch logic inside AnalyticsGrid and its existing API helpers.

11.2.3 Frontend Rendering Logic - G2 Equipment Status

Rendering is inherited from AnalyticsGrid. The standalone Analytics page passes a chartParams object containing the selected months window and optional divisionId.

Because AnalyticsGrid remains the single chart renderer, CSV export icons, skeleton loaders, responsive grid behavior and error states remain consistent between Reports and Analytics.

The page-level role is composition, not visualization implementation.

G2 Equipment Status frontend flow

```
Analytics.jsx
-> local filter state: months, divisionId
-> chartParams memo
-> <AnalyticsGrid params={chartParams} />
    -> existing chart cards
    -> existing API fetchers
    -> existing loading/error/export UX
```

11.2.4 Risks and Controls - G2 Equipment Status

Every chart can become misleading if filters are inconsistent across modules. By reusing chart parameters and AnalyticsGrid, the project reduces the risk of Reports showing one number and Analytics showing another number for the same filter.

A second risk is that charts may appear to users without the right permission. This is handled by sidebar permission mapping, protected route mapping and backend authorizeAny checks.

A third risk is performance. Since the page loads all chart cards, backend chart SQL should remain indexed and aggregate-oriented.

Decision	Expose the chart through the dedicated Analytics page.
Scope	Read-only aggregate visualization only.
Risk	Divergent filters or duplicate chart code can break trust.
Control	Reuse params, reuse grid, use protected route, preserve backend gate.

11.3.1 Chart Definition - G3 Monthly Job Trends

G3 Monthly Job Trends is part of the AnalyticsGrid surface exposed by the new /analytics page. It remains technically owned by the existing analytics chart layer, but product-wise it is now visible from the Analytics sidebar module.

The chart answers this operational question: Job request and job-card trend visibility month by month.. It helps technical users move from row inspection to pattern recognition.

The chart must remain read-only and aggregate-oriented. It should not mutate CMCMIS data, should not create audit rows and should not store generated images.

Property	Value
Chart code	G3
Chart name	Job request and job-card trend visibility month by month.
Visual type	Line/bar trend chart
Primary params	months, divisionId
Data sensitivity	Aggregate, no PII by default
Owner surface	AnalyticsGrid

11.3.2 Backend Data Contract - G3 Monthly Job Trends

The backend contract is stable: chart endpoints accept filter query parameters and return chart-ready JSON. The standalone Analytics page does not need new endpoints because the chart grid already knows how to fetch its own card datasets.

The important backend addition is not a new SQL query. It is the permission gate accepting analytics:view as a first-class permission alongside reports:view-analytics.

This keeps endpoint behavior stable and limits the change blast radius.

Decision	Reuse existing chart endpoint contract.
Scope	Only authorization semantics expand.
Risk	New endpoints would create duplicate tests and possible inconsistencies.
Control	Keep chart-specific fetch logic inside AnalyticsGrid and its existing API helpers.

11.3.3 Frontend Rendering Logic - G3 Monthly Job Trends

Rendering is inherited from AnalyticsGrid. The standalone Analytics page passes a chartParams object containing the selected months window and optional divisionId.

Because AnalyticsGrid remains the single chart renderer, CSV export icons, skeleton loaders, responsive grid behavior and error states remain consistent between Reports and Analytics.

The page-level role is composition, not visualization implementation.

G3 Monthly Job Trends frontend flow

```
Analytics.jsx
-> local filter state: months, divisionId
-> chartParams memo
-> <AnalyticsGrid params={chartParams} />
    -> existing chart cards
    -> existing API fetchers
    -> existing loading/error/export UX
```

11.3.4 Risks and Controls - G3 Monthly Job Trends

Every chart can become misleading if filters are inconsistent across modules. By reusing chart parameters and AnalyticsGrid, the project reduces the risk of Reports showing one number and Analytics showing another number for the same filter.

A second risk is that charts may appear to users without the right permission. This is handled by sidebar permission mapping, protected route mapping and backend authorizeAny checks.

A third risk is performance. Since the page loads all chart cards, backend chart SQL should remain indexed and aggregate-oriented.

Decision	Expose the chart through the dedicated Analytics page.
Scope	Read-only aggregate visualization only.
Risk	Divergent filters or duplicate chart code can break trust.
Control	Reuse params, reuse grid, use protected route, preserve backend gate.

11.4.1 Chart Definition - G4 Division-wise Jobs

G4 Division-wise Jobs is part of the AnalyticsGrid surface exposed by the new /analytics page. It remains technically owned by the existing analytics chart layer, but product-wise it is now visible from the Analytics sidebar module.

The chart answers this operational question: Workload split by division for management visibility.. It helps technical users move from row inspection to pattern recognition.

The chart must remain read-only and aggregate-oriented. It should not mutate CMCMIS data, should not create audit rows and should not store generated images.

Property	Value
Chart code	G4
Chart name	Workload split by division for management visibility.
Visual type	Bar chart
Primary params	months
Data sensitivity	Aggregate, no PII by default
Owner surface	AnalyticsGrid

11.4.2 Backend Data Contract - G4 Division-wise Jobs

The backend contract is stable: chart endpoints accept filter query parameters and return chart-ready JSON. The standalone Analytics page does not need new endpoints because the chart grid already knows how to fetch its own card datasets.

The important backend addition is not a new SQL query. It is the permission gate accepting analytics:view as a first-class permission alongside reports:view-analytics.

This keeps endpoint behavior stable and limits the change blast radius.

Decision	Reuse existing chart endpoint contract.
Scope	Only authorization semantics expand.
Risk	New endpoints would create duplicate tests and possible inconsistencies.
Control	Keep chart-specific fetch logic inside AnalyticsGrid and its existing API helpers.

11.4.3 Frontend Rendering Logic - G4 Division-wise Jobs

Rendering is inherited from AnalyticsGrid. The standalone Analytics page passes a chartParams object containing the selected months window and optional divisionId.

Because AnalyticsGrid remains the single chart renderer, CSV export icons, skeleton loaders, responsive grid behavior and error states remain consistent between Reports and Analytics.

The page-level role is composition, not visualization implementation.

G4 Division-wise Jobs frontend flow

```
Analytics.jsx
-> local filter state: months, divisionId
-> chartParams memo
-> <AnalyticsGrid params={chartParams} />
    -> existing chart cards
    -> existing API fetchers
    -> existing loading/error/export UX
```

11.4.4 Risks and Controls - G4 Division-wise Jobs

Every chart can become misleading if filters are inconsistent across modules. By reusing chart parameters and AnalyticsGrid, the project reduces the risk of Reports showing one number and Analytics showing another number for the same filter.

A second risk is that charts may appear to users without the right permission. This is handled by sidebar permission mapping, protected route mapping and backend authorizeAny checks.

A third risk is performance. Since the page loads all chart cards, backend chart SQL should remain indexed and aggregate-oriented.

Decision	Expose the chart through the dedicated Analytics page.
Scope	Read-only aggregate visualization only.
Risk	Divergent filters or duplicate chart code can break trust.
Control	Reuse params, reuse grid, use protected route, preserve backend gate.

11.5.1 Chart Definition - G5 Calibration Completion

G5 Calibration Completion is part of the AnalyticsGrid surface exposed by the new /analytics page. It remains technically owned by the existing analytics chart layer, but product-wise it is now visible from the Analytics sidebar module.

The chart answers this operational question: Completion rate and calibration closure behavior across time.. It helps technical users move from row inspection to pattern recognition.

The chart must remain read-only and aggregate-oriented. It should not mutate CMCMIS data, should not create audit rows and should not store generated images.

Property	Value
Chart code	G5
Chart name	Completion rate and calibration closure behavior across time.
Visual type	Trend or status chart
Primary params	months, divisionId
Data sensitivity	Aggregate, no PII by default
Owner surface	AnalyticsGrid

11.5.2 Backend Data Contract - G5 Calibration Completion

The backend contract is stable: chart endpoints accept filter query parameters and return chart-ready JSON. The standalone Analytics page does not need new endpoints because the chart grid already knows how to fetch its own card datasets.

The important backend addition is not a new SQL query. It is the permission gate accepting analytics:view as a first-class permission alongside reports:view-analytics.

This keeps endpoint behavior stable and limits the change blast radius.

Decision	Reuse existing chart endpoint contract.
Scope	Only authorization semantics expand.
Risk	New endpoints would create duplicate tests and possible inconsistencies.
Control	Keep chart-specific fetch logic inside AnalyticsGrid and its existing API helpers.

11.5.3 Frontend Rendering Logic - G5 Calibration Completion

Rendering is inherited from AnalyticsGrid. The standalone Analytics page passes a chartParams object containing the selected months window and optional divisionId.

Because AnalyticsGrid remains the single chart renderer, CSV export icons, skeleton loaders, responsive grid behavior and error states remain consistent between Reports and Analytics.

The page-level role is composition, not visualization implementation.

G5 Calibration Completion frontend flow

```
Analytics.jsx
-> local filter state: months, divisionId
-> chartParams memo
-> <AnalyticsGrid params={chartParams} />
    -> existing chart cards
    -> existing API fetchers
    -> existing loading/error/export UX
```

11.5.4 Risks and Controls - G5 Calibration Completion

Every chart can become misleading if filters are inconsistent across modules. By reusing chart parameters and AnalyticsGrid, the project reduces the risk of Reports showing one number and Analytics showing another number for the same filter.

A second risk is that charts may appear to users without the right permission. This is handled by sidebar permission mapping, protected route mapping and backend authorizeAny checks.

A third risk is performance. Since the page loads all chart cards, backend chart SQL should remain indexed and aggregate-oriented.

Decision	Expose the chart through the dedicated Analytics page.
Scope	Read-only aggregate visualization only.
Risk	Divergent filters or duplicate chart code can break trust.
Control	Reuse params, reuse grid, use protected route, preserve backend gate.

11.6.1 Chart Definition - G6 Job Type Distribution

G6 Job Type Distribution is part of the AnalyticsGrid surface exposed by the new /analytics page. It remains technically owned by the existing analytics chart layer, but product-wise it is now visible from the Analytics sidebar module.

The chart answers this operational question: Distribution across calibration, repair, registration or internal job categories.. It helps technical users move from row inspection to pattern recognition.

The chart must remain read-only and aggregate-oriented. It should not mutate CMCMIS data, should not create audit rows and should not store generated images.

Property	Value
Chart code	G6
Chart name	Distribution across calibration, repair, registration or internal job categories.
Visual type	Donut / bar chart
Primary params	months, divisionId
Data sensitivity	Aggregate, no PII by default
Owner surface	AnalyticsGrid

11.6.2 Backend Data Contract - G6 Job Type Distribution

The backend contract is stable: chart endpoints accept filter query parameters and return chart-ready JSON. The standalone Analytics page does not need new endpoints because the chart grid already knows how to fetch its own card datasets.

The important backend addition is not a new SQL query. It is the permission gate accepting analytics:view as a first-class permission alongside reports:view-analytics.

This keeps endpoint behavior stable and limits the change blast radius.

Decision	Reuse existing chart endpoint contract.
Scope	Only authorization semantics expand.
Risk	New endpoints would create duplicate tests and possible inconsistencies.
Control	Keep chart-specific fetch logic inside AnalyticsGrid and its existing API helpers.

11.6.3 Frontend Rendering Logic - G6 Job Type Distribution

Rendering is inherited from AnalyticsGrid. The standalone Analytics page passes a chartParams object containing the selected months window and optional divisionId.

Because AnalyticsGrid remains the single chart renderer, CSV export icons, skeleton loaders, responsive grid behavior and error states remain consistent between Reports and Analytics.

The page-level role is composition, not visualization implementation.

G6 Job Type Distribution frontend flow

```
Analytics.jsx
-> local filter state: months, divisionId
-> chartParams memo
-> <AnalyticsGrid params={chartParams} />
    -> existing chart cards
    -> existing API fetchers
    -> existing loading/error/export UX
```

11.6.4 Risks and Controls - G6 Job Type Distribution

Every chart can become misleading if filters are inconsistent across modules. By reusing chart parameters and AnalyticsGrid, the project reduces the risk of Reports showing one number and Analytics showing another number for the same filter.

A second risk is that charts may appear to users without the right permission. This is handled by sidebar permission mapping, protected route mapping and backend authorizeAny checks.

A third risk is performance. Since the page loads all chart cards, backend chart SQL should remain indexed and aggregate-oriented.

Decision	Expose the chart through the dedicated Analytics page.
Scope	Read-only aggregate visualization only.
Risk	Divergent filters or duplicate chart code can break trust.
Control	Reuse params, reuse grid, use protected route, preserve backend gate.

11.7.1 Chart Definition - G7 Engineer Workload

G7 Engineer Workload is part of the AnalyticsGrid surface exposed by the new /analytics page. It remains technically owned by the existing analytics chart layer, but product-wise it is now visible from the Analytics sidebar module.

The chart answers this operational question: Engineer-wise operational load for balancing and review.. It helps technical users move from row inspection to pattern recognition.

The chart must remain read-only and aggregate-oriented. It should not mutate CMCMIS data, should not create audit rows and should not store generated images.

Property	Value
Chart code	G7
Chart name	Engineer-wise operational load for balancing and review.
Visual type	Bar chart
Primary params	months, divisionId
Data sensitivity	Aggregate, no PII by default
Owner surface	AnalyticsGrid

11.7.2 Backend Data Contract - G7 Engineer Workload

The backend contract is stable: chart endpoints accept filter query parameters and return chart-ready JSON. The standalone Analytics page does not need new endpoints because the chart grid already knows how to fetch its own card datasets.

The important backend addition is not a new SQL query. It is the permission gate accepting analytics:view as a first-class permission alongside reports:view-analytics.

This keeps endpoint behavior stable and limits the change blast radius.

Decision	Reuse existing chart endpoint contract.
Scope	Only authorization semantics expand.
Risk	New endpoints would create duplicate tests and possible inconsistencies.
Control	Keep chart-specific fetch logic inside AnalyticsGrid and its existing API helpers.

11.7.3 Frontend Rendering Logic - G7 Engineer Workload

Rendering is inherited from AnalyticsGrid. The standalone Analytics page passes a chartParams object containing the selected months window and optional divisionId.

Because AnalyticsGrid remains the single chart renderer, CSV export icons, skeleton loaders, responsive grid behavior and error states remain consistent between Reports and Analytics.

The page-level role is composition, not visualization implementation.

G7 Engineer Workload frontend flow

```
Analytics.jsx
-> local filter state: months, divisionId
-> chartParams memo
-> <AnalyticsGrid params={chartParams} />
    -> existing chart cards
    -> existing API fetchers
    -> existing loading/error/export UX
```

11.7.4 Risks and Controls - G7 Engineer Workload

Every chart can become misleading if filters are inconsistent across modules. By reusing chart parameters and AnalyticsGrid, the project reduces the risk of Reports showing one number and Analytics showing another number for the same filter.

A second risk is that charts may appear to users without the right permission. This is handled by sidebar permission mapping, protected route mapping and backend authorizeAny checks.

A third risk is performance. Since the page loads all chart cards, backend chart SQL should remain indexed and aggregate-oriented.

Decision	Expose the chart through the dedicated Analytics page.
Scope	Read-only aggregate visualization only.
Risk	Divergent filters or duplicate chart code can break trust.
Control	Reuse params, reuse grid, use protected route, preserve backend gate.

11.8.1 Chart Definition - G8 Calibration Status Breakdown

G8 Calibration Status Breakdown is part of the AnalyticsGrid surface exposed by the new /analytics page. It remains technically owned by the existing analytics chart layer, but product-wise it is now visible from the Analytics sidebar module.

The chart answers this operational question: Calibration operational state distribution for due/overdue/completed analysis.. It helps technical users move from row inspection to pattern recognition.

The chart must remain read-only and aggregate-oriented. It should not mutate CMCMIS data, should not create audit rows and should not store generated images.

Property	Value
Chart code	G8
Chart name	Calibration operational state distribution for due/overdue/completed analysis.
Visual type	Stacked/category chart
Primary params	divisionId
Data sensitivity	Aggregate, no PII by default
Owner surface	AnalyticsGrid

11.8.2 Backend Data Contract - G8 Calibration Status Breakdown

The backend contract is stable: chart endpoints accept filter query parameters and return chart-ready JSON. The standalone Analytics page does not need new endpoints because the chart grid already knows how to fetch its own card datasets.

The important backend addition is not a new SQL query. It is the permission gate accepting analytics:view as a first-class permission alongside reports:view-analytics.

This keeps endpoint behavior stable and limits the change blast radius.

Decision	Reuse existing chart endpoint contract.
Scope	Only authorization semantics expand.
Risk	New endpoints would create duplicate tests and possible inconsistencies.
Control	Keep chart-specific fetch logic inside AnalyticsGrid and its existing API helpers.

11.8.3 Frontend Rendering Logic - G8 Calibration Status Breakdown

Rendering is inherited from AnalyticsGrid. The standalone Analytics page passes a chartParams object containing the selected months window and optional divisionId.

Because AnalyticsGrid remains the single chart renderer, CSV export icons, skeleton loaders, responsive grid behavior and error states remain consistent between Reports and Analytics.

The page-level role is composition, not visualization implementation.

G8 Calibration Status Breakdown frontend flow

```
Analytics.jsx
-> local filter state: months, divisionId
-> chartParams memo
-> <AnalyticsGrid params={chartParams} />
    -> existing chart cards
    -> existing API fetchers
    -> existing loading/error/export UX
```

11.8.4 Risks and Controls - G8 Calibration Status Breakdown

Every chart can become misleading if filters are inconsistent across modules. By reusing chart parameters and AnalyticsGrid, the project reduces the risk of Reports showing one number and Analytics showing another number for the same filter.

A second risk is that charts may appear to users without the right permission. This is handled by sidebar permission mapping, protected route mapping and backend authorizeAny checks.

A third risk is performance. Since the page loads all chart cards, backend chart SQL should remain indexed and aggregate-oriented.

Decision	Expose the chart through the dedicated Analytics page.
Scope	Read-only aggregate visualization only.
Risk	Divergent filters or duplicate chart code can break trust.
Control	Reuse params, reuse grid, use protected route, preserve backend gate.

12. Migration 420 - Logical Anatomy

Migration 420 is the database anchor of Phase 10 Part II. It adds analytics:view to permissions and grants it to the five sealed roles. It follows the same idempotent seed doctrine as previous additive migrations.

The migration has three logical blocks: permission insert, role-permission grant and verification query. Each block is explicit and safe to rerun because INSERT IGNORE is used.

The migration deliberately avoids operational tables like job request, job card or equipment master tables.

Source excerpt - migration 420

```
Write e:\SOFTWAREs By DS\cmcmis-simplified\SOFTWARE CODE\DATABASE\phase3\migrations\420__phase11s2_analytics_permission.sql
58 lines
-- =====
-- CMCMS_SIMPLIFIED - Migration 420 (Phase 11 Slice 2)
-- File:      420__phase11s2_analytics_permission.sql
-- Purpose:   Add a NEW top-level `analytics:view` permission to power
--            the dedicated /analytics sidebar page (charts-only view).
--
--            Why a new permission code, when `reports:view-analytics`
--            (Phase 10 mig 400) already gates the same chart endpoints?
--            → SEMANTIC SEPARATION. Phase 10 bundled the chart grid
--            under the Reports page; admins who deny "Reports" today
--            also lose the charts. With a standalone /analytics page
--            we want fine-grained control:
--            • A user can have analytics:view WITHOUT reports:view-*
--              (charts-only dashboard for ops staff).
--            • A user can have reports:view-* WITHOUT analytics:view
--              (reports only, no separate analytics page in nav).
--            Backend routes accept EITHER permission (authorizeAny) so
--            the existing Reports-page chart grid keeps working.
--
-- GRANT MATRIX (matches the existing reports:view-analytics matrix -
-- all 5 roles, since charts are non-sensitive aggregates with no PII):
-- SUPER_ADMIN, LAB_IN_CHARGE, LAB_ENGINEER, NORMAL_USER, VIEW_ONLY
--
-- IDEMPOTENT: INSERT IGNORE on permissions.uk_perm_code + on the
-- role_permissions PK (role_id, permission_id).
-- =====

SET NAMES utf8mb4;

-- — 420.1 Permission code (no-op if already present) —————
INSERT IGNORE INTO `permissions`
(`permission_code`, `resource`, `action`, `description`,
`is_system`, `c
VALUES
('analytics:view', 'analytics', 'view', 'View the standalone Analytics dashboard (8 chart cards)',
NOW(6));

-- — 420.2 Grant to all 5 roles (charts have no PII) —————
INSERT IGNORE INTO `role_permissions` (`role_id`, `permission_id`, `granted_at`, `granted_by`)
SELECT r.role_id, p.permission_id, NOW(6), 'BOOTSTRAP'
FROM `roles` r
CROSS JOIN `permissions` p
WHERE r.role_code IN ('SUPER_ADMIN', 'LAB_IN_CHARGE', 'LAB_ENGINEER', 'NORMAL_USER', 'VIEW_ONLY')
AND p.permission_code = 'analytics:view';

-- — 420.3 Verify —————
SELECT
p.permission_code,
COUNT(rp.role_id) AS granted_roles,
GROUP_CONCAT(r.role_code ORDER BY r.role_id) AS roles
FROM `permissions` p
```

```
LEFT JOIN `role_permissions` rp ON rp.permission_id = p.permission_id
LEFT JOIN `roles` r ON r.role_id = rp.role_id
WHERE p.permission_code = 'analytics:view'
GROUP BY p.permission_code;

SELECT '✓ Migration 420 complete – analytics:view granted to all 5 roles' AS result;
```


13. Permission Matrix - Why All Five Roles Receive analytics:view

The source transcript states that charts are non-sensitive aggregates with no PII, and therefore analytics:view is granted to all five roles. This mirrors reports:view-analytics from Phase 10.

This does not mean every raw report should be visible to every role. It means the standalone chart dashboard is considered a safe aggregate operational surface.

Future sensitive chart cards should introduce card-level backend constraints rather than weakening the current global permission semantics.

Role	analytics:view	Reason
SUPER_ADMIN	Yes	Full operational control
LAB_IN_CHARGE	Yes	Needs workload and status visibility
LAB_ENGINEER	Yes	Needs assigned/technical workload trend awareness
NORMAL_USER	Yes	Aggregate dashboard is safe and useful
VIEW_ONLY	Yes	Read-only aggregate visibility aligns with role

14. Backend Gate - authorizeAny Design

The analytics route must be usable from two products: Reports and Analytics. That means the route-level middleware should not require only analytics:view, because that would break existing Reports chart access for users who already had reports:view-analytics.

The correct backend design is an OR-style authorization function. The route accepts either analytics:view or reports:view-analytics. This is an additive permission model and preserves backward compatibility.

The no-auth case remains protected by authenticate before authorization.

Backend middleware flow

```
Request -> authenticate
        -> authorizeAny(analytics:view, reports:view-analytics)
        -> validate query params
        -> analytics controller
        -> read-only aggregate SQL
        -> JSON chart data
```

15. Analytics.jsx - Page Composition

Analytics.jsx is a composition component. It creates the page shell, title, small explanatory paragraph, mini filter bar and AnalyticsGrid mounting point.

It uses local React state for months and divisionId. It uses useMemo to produce a stable chartParams object. It also uses useQuery to fetch divisions for the dropdown.

The component is intentionally not a chart implementation file. That prevents chart duplication and keeps long-term maintenance clean.

Source excerpt - Analytics.jsx

```
Write e:\SOFTWAREs By DS\cmcmis-simplified\SOFTWARE CODE\FE\src\pages\analytics\Analytics.jsx
120 lines
// =====
// src/pages/analytics/Analytics.jsx - Standalone Analytics dashboard
// =====
// PHASE 11 SLICE 2 - splits the chart grid out of the Reports page into a
// dedicated /analytics sidebar destination. Matches the attached UI screenshot:
//
//   Analytics
//   _____
//
//   | Monthly Activity Trends | Equipment Status (donut) |
//   | Monthly Job Trends     | Division-wise Jobs     |
//   | Calibration Completion | Job Type Distribution |
//   | Engineer Workload      | Cal Status Breakdown  |
//
//
// IMPLEMENTATION
// We import the EXISTING AnalyticsGrid component from
// pages/reports/charts/AnalyticsGrid.jsx so:
//   • All 8 chart cards render with the same theme, skeleton loaders,
//   and CSV download icons.
//   • Any future chart added to AnalyticsGrid automatically appears here
//   - single source of truth.
//
// Filters are intentionally MINIMAL for the standalone page (lite filter
// bar: 6-month vs 12-month rolling window + Division). The Reports page
// continues to surface its richer filter bar because it also drives the
// detailed table view.
//
// RBAC
// Page-level gate (in App.jsx) uses `analytics:view`. The backend chart
// endpoints accept EITHER `analytics:view` or `reports:view-analytics`
// so the Reports-page chart grid keeps working without changes.
// =====

import { useMemo, useState } from 'react';
import { useQuery } from '@tanstack/react-query';

import { Layout } from '../../components/Layout.jsx';
import { AnalyticsGrid } from '../../reports/charts/AnalyticsGrid.jsx';
import { fetchDivisions } from '../../lib/api/lookups.js';

export function Analytics() {
  // — Lite filter state —
  // We keep this tiny on purpose - the Analytics page is a glance-able
  // dashboard. Heavier filtering lives on the Reports page.
  const [months, setMonths] = useState(6);
  const [divisionId, setDivisionId] = useState('');

  // Division dropdown - same source as the Reports filter bar; cached
```

```
// for an hour because divisions barely change.
const divQ = useQuery({
  queryKey: ['lookups', 'divisions'],
  queryFn: ({ signal }) => fetchDivisions(signal),
  staleTime: 60 * 60 * 1000,
});

// The chart endpoints accept `months` (1..24) and optional `divisionId`.
// We strip empty strings so the BE Zod schema doesn't reject the query.
const chartParams = useMemo(() => ({
  months,
  divisionId: divisionId || undefined,
}), [months, divisionId]);

return (
  <Layout>
    <div className="space-y-5">
      { /* — Page header — */ }
      <div className="flex items-end justify-between gap-4">
        <div>
          <h1 className="text-2xl font-semibold text-ink">Analytics</h1>
          <p className="mt-1 text-sm text-ink-soft">
            Live KPI charts across calibrations, repairs, equipment, divisions
            and engineers – aggregated directly from the database.
          </p>
        </div>

        { /* Mini filter bar – kept lightweight intentionally. */ }
        <div className="flex items-end gap-3">
          <label className="text-xs text-ink-soft">
            <span className="block mb-1">Window</span>
            <select
              value={months}
              onChange={(e) => setMonths(Number(e.target.value))}
              className="rounded-md border border-border bg-base px-3 py-2 text-sm text-ink focus:outline-none focus:ring-2 focus:ring-primary">

```

16. Filter Design - Why It Is Lightweight

The standalone Analytics page uses only a rolling time window and division selector. This is intentional. A dashboard should load quickly and support fast operational scanning.

Reports retain richer filtering because report tables and exports need detailed query control. Analytics should not become a second reporting engine.

This split keeps the product crisp: Analytics for patterns, Reports for records.

Filter	Values	Reason
Window	3, 6, 12, 24 months	Good for trend scanning
Division	All or selected division	Common operational segmentation
No detailed status filter	Deferred to Reports	Avoid dashboard overcomplexity
No export table filter	Reports ownership	Keep Analytics chart-first

17. Sidebar Entry - Discoverability Contract

Adding /analytics only to App.jsx would make the page technically reachable but not discoverable. Adding it only to the sidebar would create a broken route. Both must be changed together.

The sidebar item should depend on analytics:view so the user interface remains consistent with backend semantics.

The icon should be visually different from Reports to avoid confusion. A LineChart icon communicates chart-first behavior.

Decision	Add route and nav entry together.
Scope	App.jsx + lib/permissions.js must move as one logical commit.
Risk	Partial wiring creates broken UX or hidden pages.
Control	Verify sidebar visibility with role permissions and verify route protection.

18. Integration with Phase 10 Reports

Phase 10 already delivered Reports & Analytics with charts G1-G8. Phase 10 Part II does not reopen that sealed implementation; it extends its navigation and permission model.

The Reports page continues to show analytics grid where appropriate. The new Analytics page shows the same grid as a standalone dashboard.

This is a modular extension of the sealed phase, not a rewrite.

Capability	Reports Page	Analytics Page
Report tables	Yes	No
PDF/CSV report exports	Yes	No
AnalyticsGrid	Yes	Yes
Rich filter bar	Yes	No / lite
Sidebar label	Reports	Analytics
Primary permission	reports:*	analytics:view

19. Verification Interpretation

IMPORTANT VERIFICATION NOTE

The runner displayed 3 historical baseline check failures, but the Phase 10 Part II specific migration and verification were successful. Treat the targeted verification query and endpoint smoke test as the phase-specific proof.

The migration runner output showed the migration itself passed, while the generic runner still reported legacy baseline count checks failing. This is not a Phase 10 Part II failure; it reflects historical baseline assertions that no longer match expanded permission/user counts from later phases.

The direct verification query confirmed analytics:view is granted to all five roles. The endpoint smoke test confirmed authenticated access and unauthenticated denial.

The correct senior interpretation is: migration applied, permissions verified, endpoint protected, no-auth denied.

20. No-Regression Strategy

Because Reports still uses the same chart grid, any change to permissions could accidentally break old Reports analytics access. The authorizeAny pattern is the no-regression strategy.

Because Analytics uses AnalyticsGrid, any frontend bug fix in chart cards remains shared. The new page should not fork chart card code.

Because the migration is additive, rollback risk is low. The only new database object is an RBAC permission row and role-permission rows.

Decision	Preserve Reports behavior while adding Analytics behavior.
Scope	No chart or SQL duplication.
Risk	Permission rewiring can silently break old users.
Control	Accept old and new permission codes at backend route level.

21.1 Senior Logic Development Note 1

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 1: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.2 Senior Logic Development Note 2

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 2: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.3 Senior Logic Development Note 3

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 3: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.4 Senior Logic Development Note 4

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 4: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.5 Senior Logic Development Note 5

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 5: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.6 Senior Logic Development Note 6

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 6: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.7 Senior Logic Development Note 7

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 7: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.8 Senior Logic Development Note 8

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 8: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.9 Senior Logic Development Note 9

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 9: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.10 Senior Logic Development Note 10

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 10: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.11 Senior Logic Development Note 11

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 11: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.12 Senior Logic Development Note 12

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 12: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.13 Senior Logic Development Note 13

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 13: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.14 Senior Logic Development Note 14

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 14: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

21.15 Senior Logic Development Note 15

When developing this slice, the first question is not “which file do I edit?” but “what architectural boundary is changing?” The answer: the analytics experience is moving from nested report content to top-level navigation while preserving existing chart logic.

The second question is “where should the change be represented in the database?” Since no business data changes, the correct place is RBAC permissions, not operational tables. Migration 420 expresses this as analytics:view.

The third question is “how do we avoid regression?” The backend route must accept either the old reports-view permission or the new analytics permission, and the frontend must reuse AnalyticsGrid.

Decision	Apply logic rule 15: design from boundaries before code.
Scope	Database, backend and frontend each receive one minimal change aligned to the same product concept.
Risk	Editing files without understanding module boundaries causes scattered patches and long-term debt.
Control	Keep implementation additive, isolated, and testable through one permission query plus one endpoint smoke.

22.1 Diagram - Product Navigation Map

DIAGRAM READING RULE

Read from top to bottom. Each arrow represents responsibility transfer, not just function call order.

This diagram captures one stable relationship in the Phase 10 Part II module. It should be used during handover and future debugging to identify where a failure belongs.

The key engineering idea is that Analytics became a new product surface while the chart logic remained shared.

Product Navigation Map

```
Sidebar
  Dashboard
  Job Requests
  Job Cards
  Equipment
  Inquiry
  Reports
  Analytics <-- new Phase 10 Part II top-level entry
  Admin
```

22.2 Diagram - Analytics Page Composition

DIAGRAM READING RULE

Read from top to bottom. Each arrow represents responsibility transfer, not just function call order.

This diagram captures one stable relationship in the Phase 10 Part II module. It should be used during handover and future debugging to identify where a failure belongs.

The key engineering idea is that Analytics became a new product surface while the chart logic remained shared.

Analytics Page Composition

```
Analytics.jsx
  Header block
  Lightweight filters
    months
    divisionId
  AnalyticsGrid params
  G1..G8 chart cards
  CSV icons
  Skeleton loaders
  API fetchers
```

22.3 Diagram - Permission Flow

DIAGRAM READING RULE

Read from top to bottom. Each arrow represents responsibility transfer, not just function call order.

This diagram captures one stable relationship in the Phase 10 Part II module. It should be used during handover and future debugging to identify where a failure belongs.

The key engineering idea is that Analytics became a new product surface while the chart logic remained shared.

Permission Flow

```
Login
-> token contains permissions
-> sidebar checks analytics:view
-> route guard checks analytics:view
-> backend accepts analytics:view OR reports:view-analytics
-> chart JSON returned
```

22.4 Diagram - Backward Compatibility Flow

DIAGRAM READING RULE

Read from top to bottom. Each arrow represents responsibility transfer, not just function call order.

This diagram captures one stable relationship in the Phase 10 Part II module. It should be used during handover and future debugging to identify where a failure belongs.

The key engineering idea is that Analytics became a new product surface while the chart logic remained shared.

Backward Compatibility Flow

```
Reports page user
-> has reports:view-analytics
-> calls /api/v1/analytics/*
-> authorizeAny passes
-> existing Reports chart grid still works
```

22.5 Diagram - New Analytics Flow

DIAGRAM READING RULE

Read from top to bottom. Each arrow represents responsibility transfer, not just function call order.

This diagram captures one stable relationship in the Phase 10 Part II module. It should be used during handover and future debugging to identify where a failure belongs.

The key engineering idea is that Analytics became a new product surface while the chart logic remained shared.

New Analytics Flow

```
Analytics page user
-> has analytics:view
-> opens /analytics
-> AnalyticsGrid fetches chart endpoints
-> backend accepts analytics:view
-> same chart cards render
```

22.6 Diagram - No Duplication Flow

DIAGRAM READING RULE

Read from top to bottom. Each arrow represents responsibility transfer, not just function call order.

This diagram captures one stable relationship in the Phase 10 Part II module. It should be used during handover and future debugging to identify where a failure belongs.

The key engineering idea is that Analytics became a new product surface while the chart logic remained shared.

No Duplication Flow

```
Reports AnalyticsGrid instance
      \
      //      shared component -> chart cards -> API helpers
Analytics page AnalyticsGrid instance
```

22.7 Diagram - Failure Flow: No Token

DIAGRAM READING RULE

Read from top to bottom. Each arrow represents responsibility transfer, not just function call order.

This diagram captures one stable relationship in the Phase 10 Part II module. It should be used during handover and future debugging to identify where a failure belongs.

The key engineering idea is that Analytics became a new product surface while the chart logic remained shared.

Failure Flow: No Token

```
Request /api/v1/analytics/monthly-jobs
-> authenticate fails
-> 401
-> no controller execution
-> no data leakage
```

22.8 Diagram - Failure Flow: Missing Permission

DIAGRAM READING RULE

Read from top to bottom. Each arrow represents responsibility transfer, not just function call order.

This diagram captures one stable relationship in the Phase 10 Part II module. It should be used during handover and future debugging to identify where a failure belongs.

The key engineering idea is that Analytics became a new product surface while the chart logic remained shared.

Failure Flow: Missing Permission

Authenticated user without analytics/view or reports/view-analytics

- > authenticate passes
- > authorizeAny fails
- > 403
- > chart data not returned

23.1 Technical Table - RBAC Matrix

This table provides a compact operational reference for the Phase 10 Part II analytics sidebar module. It is intended for fast revision and maintenance decisions.

Use the matrix to confirm that database, backend, frontend and user-facing behavior remain aligned.

Role	analytics:view	reports:view-analytics	Sidebar Analytics	Reports Analytics
SUPER_ADMIN	Yes	Yes	Visible	Visible
LAB_IN_CHARGE	Yes	Yes	Visible	Visible
LAB_ENGINEER	Yes	Yes	Visible	Visible
NORMAL_USER	Yes	Yes	Visible	Visible
VIEW_ONLY	Yes	Yes	Visible	Visible

23.2 Technical Table - Layer Responsibility Matrix

This table provides a compact operational reference for the Phase 10 Part II analytics sidebar module. It is intended for fast revision and maintenance decisions.

Use the matrix to confirm that database, backend, frontend and user-facing behavior remain aligned.

Layer	Responsibility	Phase 10 Part II Change
Database	Permission catalog and grants	Migration 420
Backend	Authenticated analytics APIs	authorizeAny gate
Frontend Page	Analytics user experience	Analytics.jsx
Routing	Protected page path	/analytics route
Sidebar	Discoverability	Analytics nav item
Chart Grid	Visualization body	Reused, not copied

23.3 Technical Table - Endpoint Access Matrix

This table provides a compact operational reference for the Phase 10 Part II analytics sidebar module. It is intended for fast revision and maintenance decisions.

Use the matrix to confirm that database, backend, frontend and user-facing behavior remain aligned.

Endpoint Family	Old Permission	New Permission	Status
/api/v1/analytics/*	reports:view-analytics	analytics:view	Either accepted
/reports	reports permissions	Not required	Unchanged
/analytics	Not required	analytics:view	New page
lookups/divisions	Existing lookup access	Existing auth	Used by filter

23.4 Technical Table - Smoke Matrix

This table provides a compact operational reference for the Phase 10 Part II analytics sidebar module. It is intended for fast revision and maintenance decisions.

Use the matrix to confirm that database, backend, frontend and user-facing behavior remain aligned.

Test	Result	Meaning
Migration 420	Pass	Migration file executed
Permission query	All 5 roles	RBAC seeded
SA equipment-status	200 rows=2	Authenticated endpoint works
No-auth monthly-jobs	401	Authentication enforced
Frontend build	Expected clean	No syntax break

23.5 Technical Table - Risk Matrix

This table provides a compact operational reference for the Phase 10 Part II analytics sidebar module. It is intended for fast revision and maintenance decisions.

Use the matrix to confirm that database, backend, frontend and user-facing behavior remain aligned.

Risk	Cause	Control
Duplicate chart logic	Copying chart cards into Analytics	Reuse AnalyticsGrid
Reports regression	Backend only checks analytics:view	authorizeAny
Hidden route	No sidebar entry	permissions.js nav item
Broken route	Sidebar without App route	App.jsx route
Over-filtered dashboard	Trying to rebuild Reports	Keep filters lite

24.1 Code Appendix - Migration 420 SQL (1/3)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Migration 420 SQL excerpt

```
-- =====
-- CMCMS_SIMPLIFIED - Migration 420 (Phase 11 Slice 2)
-- File:      420__phase11s2_analytics_permission.sql
-- Purpose:   Add a NEW top-level `analytics:view` permission to power
--            the dedicated /analytics sidebar page (charts-only view).
--
--            Why a new permission code, when `reports:view-analytics`
--            (Phase 10 mig 400) already gates the same chart endpoints?
--            → SEMANTIC SEPARATION. Phase 10 bundled the chart grid
--            under the Reports page; admins who deny "Reports" today
--            also lose the charts. With a standalone /analytics page
--            we want fine-grained control:
--            • A user can have analytics:view WITHOUT reports:view-*
--              (charts-only dashboard for ops staff).
--            • A user can have reports:view-* WITHOUT analytics:view
--              (reports only, no separate analytics page in nav).
--            Backend routes accept EITHER permission (authorizeAny) so
--            the existing Reports-page chart grid keeps working.
--
-- GRANT MATRIX (matches the existing reports:view-analytics matrix -
-- all 5 roles, since charts are non-sensitive aggregates with no PII):
-- SUPER_ADMIN, LAB_IN_CHARGE, LAB_ENGINEER, NORMAL_USER, VIEW_ONLY
--
-- IDEMPOTENT: INSERT IGNORE on permissions.uk_perm_code + on the
-- role_permissions PK (role_id, permission_id).
-- =====

SET NAMES utf8mb4;
```

24.2 Code Appendix - Migration 420 SQL (2/3)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Migration 420 SQL excerpt

```
-- — 420.1 Permission code (no-op if already present) —————
INSERT IGNORE INTO `permissions`
  (`permission_code`, `resource`, `action`, `description`,
  `is_system`, `c
VALUES
  ('analytics:view', 'analytics', 'view', 'View the standalone Analytics dashboard (8 chart cards)', 1,
  NOW(6));

-- — 420.2 Grant to all 5 roles (charts have no PII) —————
INSERT IGNORE INTO `role_permissions` (`role_id`, `permission_id`, `granted_at`, `granted_by`)
SELECT r.role_id, p.permission_id, NOW(6), 'BOOTSTRAP'
  FROM `roles` r
  CROSS JOIN `permissions` p
 WHERE r.role_code IN ('SUPER_ADMIN', 'LAB_IN_CHARGE', 'LAB_ENGINEER', 'NORMAL_USER', 'VIEW_ONLY')
    AND p.permission_code = 'analytics:view';

-- — 420.3 Verify —————
SELECT
  p.permission_code,
  COUNT(rp.role_id) AS granted_roles,
  GROUP_CONCAT(r.role_code ORDER BY r.role_id) AS roles
  FROM `permissions` p
  LEFT JOIN `role_permissions` rp ON rp.permission_id = p.permission_id
  LEFT JOIN `roles` r ON r.role_id = rp.role_id
 WHERE p.permission_code = 'analytics:view'
 GROUP BY p.permission_code;
```

24.3 Code Appendix - Migration 420 SQL (3/3)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Migration 420 SQL excerpt

```
SELECT '✓ Migration 420 complete – analytics:view granted to all 5 roles' AS result;
```


24.4 Code Appendix - Analytics.jsx React Page (1/5)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Analytics.jsx React Page excerpt

```
// =====
// src/pages/analytics/Analytics.jsx - Standalone Analytics dashboard
// =====
// PHASE 11 SLICE 2 - splits the chart grid out of the Reports page into a
// dedicated /analytics sidebar destination. Matches the attached UI screenshot:
//
//   Analytics
//   _____
//
//   | Monthly Activity Trends | Equipment Status (donut) |
//   | Monthly Job Trends     | Division-wise Jobs      |
//   | Calibration Completion  | Job Type Distribution |
//   | Engineer Workload       | Cal Status Breakdown  |
//
//
// IMPLEMENTATION
// We import the EXISTING AnalyticsGrid component from
// pages/reports/charts/AnalyticsGrid.jsx so:
//   • All 8 chart cards render with the same theme, skeleton loaders,
//     and CSV download icons.
//   • Any future chart added to AnalyticsGrid automatically appears here
//     - single source of truth.
//
// Filters are intentionally MINIMAL for the standalone page (lite filter
// bar: 6-month vs 12-month rolling window + Division). The Reports page
```

24.5 Code Appendix - Analytics.jsx React Page (2/5)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Analytics.jsx React Page excerpt

```
// continues to surface its richer filter bar because it also drives the
// detailed table view.
//
// RBAC
// Page-level gate (in App.jsx) uses `analytics:view`. The backend chart
// endpoints accept EITHER `analytics:view` or `reports:view-analytics`
// so the Reports-page chart grid keeps working without changes.
// =====

import { useMemo, useState } from 'react';
import { useQuery } from '@tanstack/react-query';

import { Layout } from '../../components/Layout.jsx';
import { AnalyticsGrid } from '../../reports/charts/AnalyticsGrid.jsx';
import { fetchDivisions } from '../../lib/api/lookups.js';

export function Analytics() {
  // — Lite filter state —————
  // We keep this tiny on purpose – the Analytics page is a glance-able
  // dashboard. Heavier filtering lives on the Reports page.
  const [months, setMonths] = useState(6);
  const [divisionId, setDivisionId] = useState('');

  // Division dropdown – same source as the Reports filter bar; cached
  // for an hour because divisions barely change.
  const divQ = useQuery({
    queryKey: ['lookups', 'divisions'],
    queryFn: ({ signal }) => fetchDivisions(signal),
```

24.6 Code Appendix - Analytics.jsx React Page (3/5)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Analytics.jsx React Page excerpt

```

    staleTime: 60 * 60 * 1000,
  });

  // The chart endpoints accept `months` (1..24) and optional `divisionId`.
  // We strip empty strings so the BE Zod schema doesn't reject the query.
  const chartParams = useMemo(() => ({
    months,
    divisionId: divisionId || undefined,
  }), [months, divisionId]);

  return (
    <Layout>
      <div className="space-y-5">
        { /* — Page header — */ }
        <div className="flex items-end justify-between gap-4">
          <div>
            <h1 className="text-2xl font-semibold text-ink">Analytics</h1>
            <p className="mt-1 text-sm text-ink-soft">
              Live KPI charts across calibrations, repairs, equipment, divisions
              and engineers — aggregated directly from the database.
            </p>
          </div>
        </div>

        { /* Mini filter bar — kept lightweight intentionally. */ }
        <div className="flex items-end gap-3">
          <label className="text-xs text-ink-soft">
            <span className="block mb-1">Window</span>
            <select

```

24.7 Code Appendix - Analytics.jsx React Page (4/5)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Analytics.jsx React Page excerpt

```

        value={months}
        onChange={(e) => setMonths(Number(e.target.value))}
        className="rounded-md border border-border bg-base px-3 py-2 text-sm text-ink focus:outline-none
focus:ring-2 focus:
    >
    <option value={3}>Last 3 months</option>
    <option value={6}>Last 6 months</option>
    <option value={12}>Last 12 months</option>
    <option value={24}>Last 24 months</option>
  </select>
</label>
<label className="text-xs text-ink-soft">
  <span className="block mb-1">Division</span>
  <select
    value={divisionId}
    onChange={(e) => setDivisionId(e.target.value ? Number(e.target.value) : '')}
    disabled={divQ.isLoading}
    className="rounded-md border border-border bg-base px-3 py-2 text-sm text-ink focus:outline-none
focus:ring-2 focus:
    >
    <option value="">All Divisions</option>
    {(divQ.data || []).map((d) => (
      <option key={d.id} value={d.id}>
        {d.code}{d.name ? ` - ${d.name}` : ''}
      </option>
    ))}
  </select>
</label>
</div>
</div>

```

24.8 Code Appendix - Analytics.jsx React Page (5/5)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Analytics.jsx React Page excerpt

```
    { /* — Chart grid – reused from Reports page ————— */ }  
    <AnalyticsGrid params={chartParams} />  
  </div>  
</Layout>  
);  
}
```

24.9 Code Appendix - Permission Verification Query (1/1)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Permission Verification Query excerpt

```
Bash Verify analytics:view perm
IN
cd "/e/SOFTWAREs By DS/cmcmis-simplified/SOFTWARE CODE/BE" && node -e "
const mysql = require('mysql2/promise');
(async () => {
  const conn = await mysql.createConnection({ host:'localhost', user:'root', database:'final' });
  const [rows] = await conn.query("\SELECT p.permission_code, GROUP_CONCAT(r.role_code ORDER BY r.role_id) AS roles FROM
permisio
  rows.forEach(r => console.log(r.permission_code, '→', r.roles));
  await conn.end();
})();
"

OUT
analytics:view → SUPER_ADMIN,LAB_IN_CHARGE,LAB_ENGINEER,NORMAL_USER,VIEW_ONLY
```

24.10 Code Appendix - Endpoint Smoke Test (1/1)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Endpoint Smoke Test excerpt

```
Bash Spot-test analytics endpoint with new gate
IN
sleep 2 && cd "/e/SOFTWAREs By DS/cmcmis-simplified/SOFTWARE CODE/BE" && node -e "
(async () => {
  const r = await fetch('http://localhost:3000/api/v1/auth/login', { method:'POST', headers:{'content-
type':'application/json'}, b
  const token = (await r.json()).data.accessToken;
  // Hit a chart endpoint as SA (who now has both analytics:view AND reports:view-analytics)
  const r2 = await fetch('http://localhost:3000/api/v1/analytics/equipment-status?months=6', {
headers:{Authorization:'Bearer '+to
  const j2 = await r2.json();
  console.log('SA → /analytics/equipment-status:', r2.status, 'rows=', j2.data?.length);
  // Verify no-auth still 401
  const r3 = await fetch('http://localhost:3000/api/v1/analytics/monthly-jobs');
  console.log('no-auth → 401?', r3.status);
})();
"

OUT
SA → /analytics/equipment-status: 200 rows= 2
no-auth → 401? 401
```

24.11 Code Appendix - Full Source Transcript (1/10)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Full Source Transcript excerpt

```
I'll add an "Analytics" sidebar entry that opens a dedicated charts-only page using the existing AnalyticsGrid component,
plus a n

Update Todos

Migration 420: seed analytics:view permission (additive, all 5 roles)

Backend: relax analytics route gate to authorizeAny(analytics:view, reports:view-analytics)

Frontend: new pages/analytics/Analytics.jsx (charts-only, reuses AnalyticsGrid)

Frontend: wire /analytics route in App.jsx + new sidebar entry in lib/permissions.js

Verify migration + endpoint smoke + FE build

Update Phase 11 memory note
Write e:\SOFTWAREs By DS\cmcmis-simplified\SOFTWARE CODE\DATABASE\phase3\migrations\420__phase11s2_analytics_permission.sql
58 lines
-- =====
-- CMCMS_SIMPLIFIED - Migration 420 (Phase 11 Slice 2)
-- File:      420__phase11s2_analytics_permission.sql
-- Purpose:   Add a NEW top-level `analytics:view` permission to power
--            the dedicated /analytics sidebar page (charts-only view).
--
--            Why a new permission code, when `reports:view-analytics`
--            (Phase 10 mig 400) already gates the same chart endpoints?
--            → SEMANTIC SEPARATION. Phase 10 bundled the chart grid
--            under the Reports page; admins who deny "Reports" today
--            also lose the charts. With a standalone /analytics page
```


24.12 Code Appendix - Full Source Transcript (2/10)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Full Source Transcript excerpt

```
--      we want fine-grained control:
--      • A user can have analytics:view WITHOUT reports:view-*
--        (charts-only dashboard for ops staff).
--      • A user can have reports:view-* WITHOUT analytics:view
--        (reports only, no separate analytics page in nav).
--      Backend routes accept EITHER permission (authorizeAny) so
--      the existing Reports-page chart grid keeps working.
--
-- GRANT MATRIX (matches the existing reports:view-analytics matrix -
-- all 5 roles, since charts are non-sensitive aggregates with no PII):
--     SUPER_ADMIN, LAB_IN_CHARGE, LAB_ENGINEER, NORMAL_USER, VIEW_ONLY
--
-- IDEMPOTENT: INSERT IGNORE on permissions.uk_perm_code + on the
-- role_permissions PK (role_id, permission_id).
-- =====

SET NAMES utf8mb4;

-- — 420.1 Permission code (no-op if already present) —————
INSERT IGNORE INTO `permissions`
  (`permission_code`, `resource`, `action`, `description`,
  `is_system`, `c
VALUES
  ('analytics:view', 'analytics', 'view', 'View the standalone Analytics dashboard (8 chart cards)',
  NOW(6));
1,

-- — 420.2 Grant to all 5 roles (charts have no PII) —————
INSERT IGNORE INTO `role_permissions` (`role_id`, `permission_id`, `granted_at`, `granted_by`)
SELECT r.role_id, p.permission_id, NOW(6), 'BOOTSTRAP'
```

24.13 Code Appendix - Full Source Transcript (3/10)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Full Source Transcript excerpt

```
FROM `roles` r
CROSS JOIN `permissions` p
WHERE r.role_code IN ('SUPER_ADMIN','LAB_IN_CHARGE','LAB_ENGINEER','NORMAL_USER','VIEW_ONLY')
AND p.permission_code = 'analytics:view';

-- — 420.3 Verify —————
SELECT
  p.permission_code,
  COUNT(rp.role_id) AS granted_roles,
  GROUP_CONCAT(r.role_code ORDER BY r.role_id) AS roles
FROM `permissions` p
LEFT JOIN `role_permissions` rp ON rp.permission_id = p.permission_id
LEFT JOIN `roles` r ON r.role_id = rp.role_id
WHERE p.permission_code = 'analytics:view'
GROUP BY p.permission_code;

SELECT '✓ Migration 420 complete – analytics:view granted to all 5 roles' AS result;
Bash Run migration 420
IN
cd "/e/SOFTWAREs By DS/cmcmis-simplified/SOFTWARE CODE/DATABASE/phase3/runner" && node run-migrations.js 2>&1 | grep -E
"(420|anal

OUT
[1m▶ 420__phase11s2_analytics_permission.sql [0m
[32m✓ 420__phase11s2_analytics_permission.sql (11ms) [0m
[31mX 3 of 14 checks FAILED. Investigate before declaring done. [0m
Bash Verify analytics:view perm
IN
```

24.14 Code Appendix - Full Source Transcript (4/10)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Full Source Transcript excerpt

```
cd "/e/SOFTWAREs By DS/cmcmis-simplified/SOFTWARE CODE/BE" && node -e "
const mysql = require('mysql2/promise');
(async () => {
  const conn = await mysql.createConnection({ host:'localhost', user:'root', database:'final' });
  const [rows] = await conn.query(\"SELECT p.permission_code, GROUP_CONCAT(r.role_code ORDER BY r.role_id) AS roles FROM
permisio
  rows.forEach(r => console.log(r.permission_code, '→', r.roles));
  await conn.end();
})();
"

OUT
analytics:view → SUPER_ADMIN,LAB_IN_CHARGE,LAB_ENGINEER,NORMAL_USER,VIEW_ONLY
Permission seeded for all 5 roles. Now relax the BE gate so the chart endpoints accept either permission.

Edit e:\SOFTWAREs By DS\cmcmis-simplified\SOURCE CODE\BE\src\modules\analytics\analytics.routes.js
Added 9 lines
Now build the FE Analytics page. Reuse the existing AnalyticsGrid so all 8 charts render with zero duplication.

Bash Create FE analytics dir
IN
mkdir -p "/e/SOFTWAREs By DS/cmcmis-simplified/SOFTWARE CODE/FE/src/pages/analytics"

OUT
(Bash completed with no output)
Write e:\SOFTWAREs By DS\cmcmis-simplified\SOURCE CODE\FE\src\pages\analytics\Analytics.jsx
120 lines
// =====
// src/pages/analytics/Analytics.jsx - Standalone Analytics dashboard
```

24.15 Code Appendix - Full Source Transcript (5/10)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Full Source Transcript excerpt

```
// -----
// PHASE 11 SLICE 2 – splits the chart grid out of the Reports page into a
// dedicated /analytics sidebar destination. Matches the attached UI screenshot:
//
//   Analytics
//   _____
//   | Monthly Activity Trends | Equipment Status (donut) |
//   | Monthly Job Trends     | Division-wise Jobs      |
//   | Calibration Completion | Job Type Distribution |
//   | Engineer Workload      | Cal Status Breakdown  |
//
// IMPLEMENTATION
// We import the EXISTING AnalyticsGrid component from
// pages/reports/charts/AnalyticsGrid.jsx so:
//   • All 8 chart cards render with the same theme, skeleton loaders,
//     and CSV download icons.
//   • Any future chart added to AnalyticsGrid automatically appears here
//     – single source of truth.
//
// Filters are intentionally MINIMAL for the standalone page (lite filter
// bar: 6-month vs 12-month rolling window + Division). The Reports page
// continues to surface its richer filter bar because it also drives the
// detailed table view.
```

24.16 Code Appendix - Full Source Transcript (6/10)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Full Source Transcript excerpt

```
//
// RBAC
// Page-level gate (in App.jsx) uses `analytics:view`. The backend chart
// endpoints accept EITHER `analytics:view` or `reports:view-analytics`
// so the Reports-page chart grid keeps working without changes.
// =====

import { useMemo, useState } from 'react';
import { useQuery } from '@tanstack/react-query';

import { Layout } from '../../components/Layout.jsx';
import { AnalyticsGrid } from '../../reports/charts/AnalyticsGrid.jsx';
import { fetchDivisions } from '../../lib/api/lookups.js';

export function Analytics() {
  // — Lite filter state —————
  // We keep this tiny on purpose – the Analytics page is a glance-able
  // dashboard. Heavier filtering lives on the Reports page.
  const [months, setMonths] = useState(6);
  const [divisionId, setDivisionId] = useState('');

  // Division dropdown – same source as the Reports filter bar; cached
  // for an hour because divisions barely change.
  const divQ = useQuery({
    queryKey: ['lookups', 'divisions'],
    queryFn: ({ signal }) => fetchDivisions(signal),
    staleTime: 60 * 60 * 1000,
  });
}
```

24.17 Code Appendix - Full Source Transcript (7/10)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Full Source Transcript excerpt

```
// The chart endpoints accept `months` (1..24) and optional `divisionId`.
// We strip empty strings so the BE Zod schema doesn't reject the query.
const chartParams = useMemo(() => ({
  months,
  divisionId: divisionId || undefined,
}), [months, divisionId]);

return (
  <Layout>
    <div className="space-y-5">
      {/* — Page header ————— */}
      <div className="flex items-end justify-between gap-4">
        <div>
          <h1 className="text-2xl font-semibold text-ink">Analytics</h1>
          <p className="mt-1 text-sm text-ink-soft">
            Live KPI charts across calibrations, repairs, equipment, divisions
            and engineers – aggregated directly from the database.
          </p>
        </div>

        {/* Mini filter bar – kept lightweight intentionally. */}
        <div className="flex items-end gap-3">
          <label className="text-xs text-ink-soft">
            <span className="block mb-1">Window</span>
            <select
              value={months}
              onChange={(e) => setMonths(Number(e.target.value))}>
```

24.18 Code Appendix - Full Source Transcript (8/10)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Full Source Transcript excerpt

```

        className="rounded-md border border-border bg-base px-3 py-2 text-sm text-ink focus:outline-none
focus:ring-2 focus:
    >
    <option value={3}>Last 3 months</option>
    <option value={6}>Last 6 months</option>
    <option value={12}>Last 12 months</option>
    <option value={24}>Last 24 months</option>
  </select>
</label>
<label className="text-xs text-ink-soft">
  <span className="block mb-1">Division</span>
  <select
    value={divisionId}
    onChange={(e) => setDivisionId(e.target.value ? Number(e.target.value) : '')}
    disabled={divQ.isLoading}
    className="rounded-md border border-border bg-base px-3 py-2 text-sm text-ink focus:outline-none
focus:ring-2 focus:
    >
    <option value="">All Divisions</option>
    {(divQ.data || []).map((d) => (
      <option key={d.id} value={d.id}>
        {d.code}{d.name ? ` - ${d.name}` : ''}
      </option>
    ))}
  </select>
</label>
</div>
</div>

{ /* — Chart grid — reused from Reports page ————— */ }

```

24.19 Code Appendix - Full Source Transcript (9/10)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Full Source Transcript excerpt

```

    <AnalyticsGrid params={chartParams} />
  </div>
</Layout>
);
}

```

Now wire the route in App.jsx and the sidebar entry in permissions.js.

Read e:\SOFTWAREs By DS\cmcmis-simplified\SOFTWARE CODE\FE\src\App.jsx (lines 164-193)

Update Todos

Migration 420: seed analytics:view permission (additive, all 5 roles)

Backend: relax analytics route gate to authorizeAny(analytics:view, reports:view-analytics)

Frontend: new pages/analytics/Analytics.jsx (charts-only, reuses AnalyticsGrid)

Frontend: wire /analytics route in App.jsx + new sidebar entry in lib/permissions.js

Verify migration + endpoint smoke + FE build

Update Phase 11 memory note

Edit e:\SOFTWAREs By DS\cmcmis-simplified\SOFTWARE CODE\FE\src\App.jsx

Added 2 lines

Edit e:\SOFTWAREs By DS\cmcmis-simplified\SOFTWARE CODE\FE\src\App.jsx

Added 12 lines

Now add the sidebar nav entry. Use LineChart icon to differentiate from Reports' BarChart3.

Edit e:\SOFTWAREs By DS\cmcmis-simplified\SOFTWARE CODE\FE\src\lib\permissions.js

24.20 Code Appendix - Full Source Transcript (10/10)

CODE APPENDIX RULE

Use code appendices to verify implementation intent. Do not fork this logic into parallel modules.

This appendix preserves implementation evidence from the uploaded transcript. The code is included for auditability and future reference, but the primary architecture is explained in the earlier sections.

Read this as source context, not as a request to duplicate logic elsewhere.

Full Source Transcript excerpt

```
Added 1 line
Edit e:\SOFTWAREs By DS\cmcmis-simplified\SOFTWARE CODE\FE\src\lib\permissions.js
Added 4 lines
Verify end-to-end: BE endpoint still works for users with only analytics:view, FE builds clean.

Bash Spot-test analytics endpoint with new gate
IN
sleep 2 && cd "/e/SOFTWAREs By DS/cmcmis-simplified/SOFTWARE CODE/BE" && node -e "
(async () => {
  const r = await fetch('http://localhost:3000/api/v1/auth/login', { method:'POST', headers:{'content-
type':'application/json'}}, b
  const token = (await r.json()).data.accessToken;
  // Hit a chart endpoint as SA (who now has both analytics:view AND reports:view-analytics)
  const r2 = await fetch('http://localhost:3000/api/v1/analytics/equipment-status?months=6', {
headers:{Authorization:'Bearer '+to
  const j2 = await r2.json();
  console.log('SA → /analytics/equipment-status:', r2.status, 'rows=', j2.data?.length);
  // Verify no-auth still 401
  const r3 = await fetch('http://localhost:3000/api/v1/analytics/monthly-jobs');
  console.log('no-auth → 401?', r3.status);
})();
"

OUT
SA → /analytics/equipment-status: 200 rows= 2
no-auth → 401? 401
```

25.1 DS Senior Playbook - Module Boundary Thinking

Module Boundary Thinking is a repeatable engineering lens for this analytics-sidebar module. The slice is small, but it demonstrates how production systems should evolve without breaking sealed modules.

Every future enhancement must ask whether it changes product navigation, permission semantics, data shape, visualization rendering or backend aggregation. Mixing these concerns leads to fragile patches.

For this phase, the safest path was additive RBAC, shared backend endpoints, reused chart components and minimal new UI composition. That pattern should become the template for small module elevations in CMCMIS_SIMPLIFIED.

A future developer should prefer one verified route, one permission contract, one reusable chart component and one clearly documented sidebar entry over multiple duplicated implementation paths.

Decision	Use Module Boundary Thinking as a review gate.
Scope	Apply the gate to Analytics, Reports and future dashboard modules.
Risk	Skipping this gate can introduce duplicate APIs, fragmented permissions or invisible pages.
Control	Document the decision, verify endpoint behavior and keep the module sealed after acceptance.

25.2 DS Senior Playbook - RBAC Evolution

RBAC Evolution is a repeatable engineering lens for this analytics-sidebar module. The slice is small, but it demonstrates how production systems should evolve without breaking sealed modules.

Every future enhancement must ask whether it changes product navigation, permission semantics, data shape, visualization rendering or backend aggregation. Mixing these concerns leads to fragile patches.

For this phase, the safest path was additive RBAC, shared backend endpoints, reused chart components and minimal new UI composition. That pattern should become the template for small module elevations in CMCMIS_SIMPLIFIED.

A future developer should prefer one verified route, one permission contract, one reusable chart component and one clearly documented sidebar entry over multiple duplicated implementation paths.

Decision	Use RBAC Evolution as a review gate.
Scope	Apply the gate to Analytics, Reports and future dashboard modules.
Risk	Skipping this gate can introduce duplicate APIs, fragmented permissions or invisible pages.
Control	Document the decision, verify endpoint behavior and keep the module sealed after acceptance.

Final Locked Seal - Phase 10 Part II Analytics Sidebar Module

DS FINAL LOCK

Phase 10 Part II Analytics Sidebar New Module is documentation-sealed. Future changes must preserve reuse-first AnalyticsGrid doctrine and additive RBAC semantics.

Phase 10 Part II is sealed as an additive analytics navigation and RBAC enhancement. The system now supports a dedicated Analytics sidebar module using analytics:view while preserving existing Reports chart access through reports:view-analytics.

The new page does not duplicate chart components. It reuses AnalyticsGrid, applies lightweight filters and keeps the dashboard chart experience consistent with Reports.

The database change is migration 420. The backend change is authorizeAny over the old and new permissions. The frontend change is Analytics.jsx plus route/sidebar wiring. Verification confirms authenticated analytics access and unauthenticated denial.

Seal Item	Status
Migration 420	SEALED
analytics:view permission	SEALED
Backend authorizeAny gate	SEALED
Analytics.jsx standalone page	SEALED
App route /analytics	SEALED
Sidebar entry	SEALED
Endpoint smoke	SEALED

